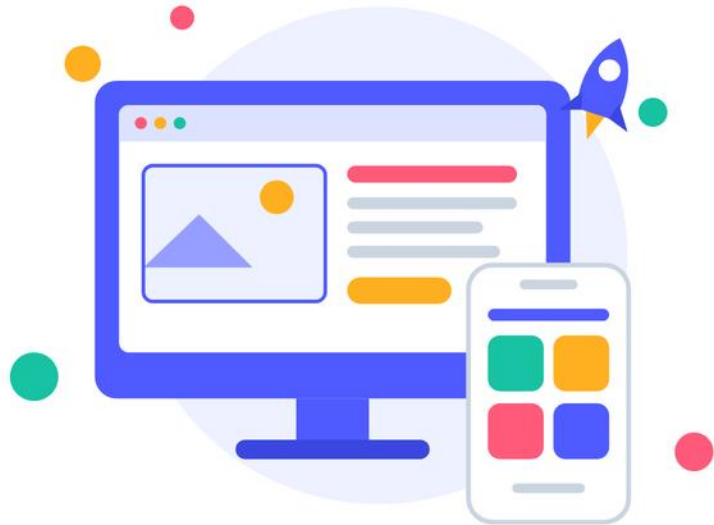


WEEK 03 · FLOW · POSITION · FLEXBOX

WHERE THE BOXES GO

Last week you learned to style a box. Today you learn to put one somewhere.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



WHERE WE ARE

Week 3 of 12. You have a page with a design language and no layout.

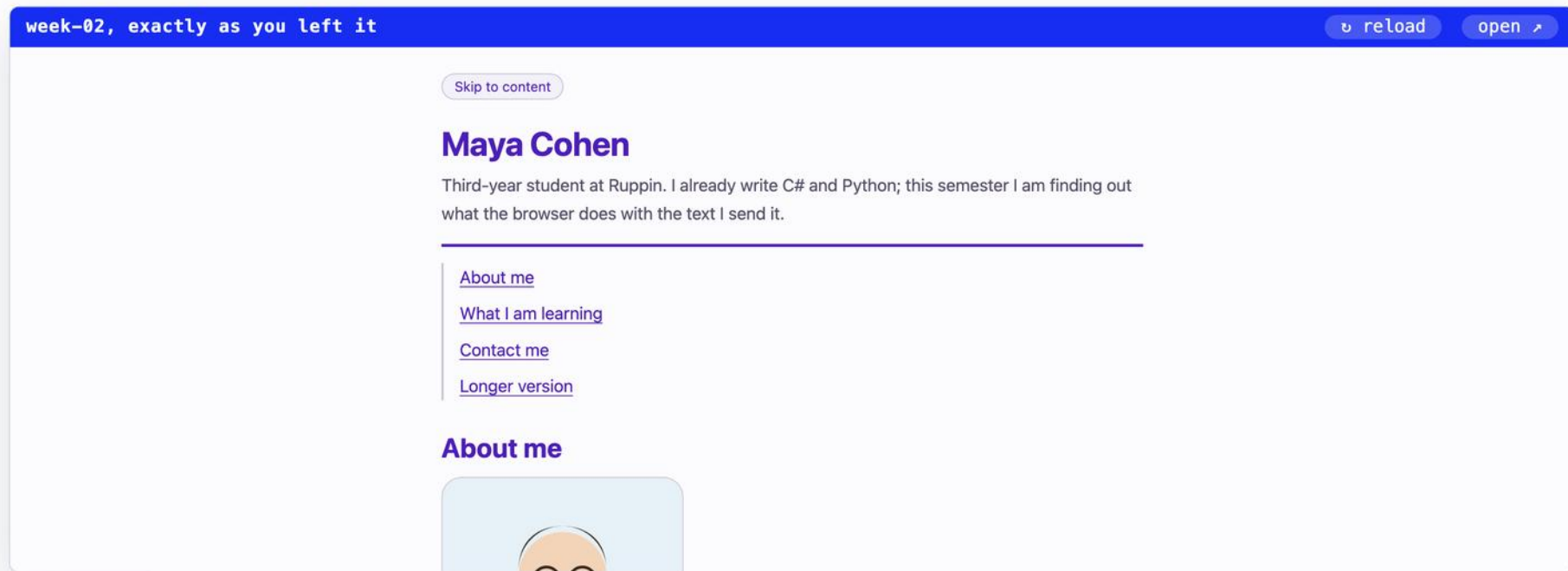
AFTER TODAY YOU CAN

Explain why a page looks the way it does before you touch it, place a box deliberately, and build a navbar, a hero and a card row that respond on their own.

WHAT THIS UNLOCKS

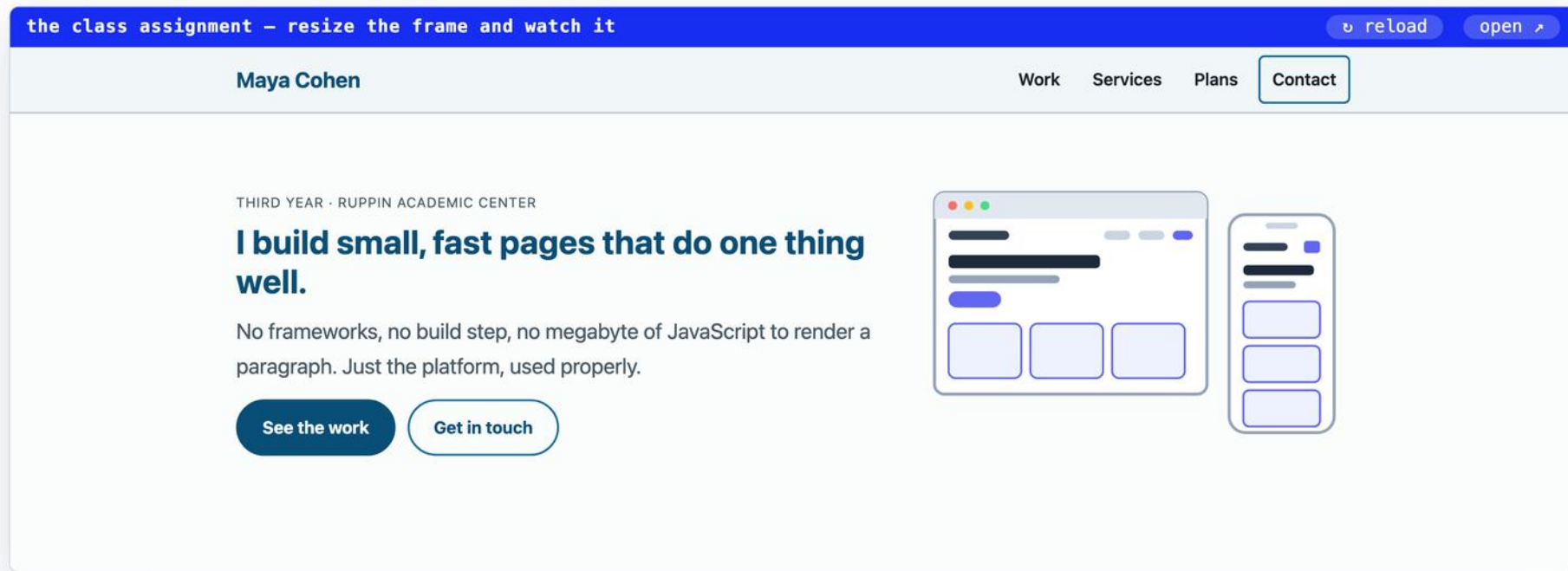
Week 4 is Grid, and the decision rule "Grid for the page, Flex inside the component" only means something once you know what Flex is.

THIS IS EVERYTHING YOU CAN BUILD RIGHT NOW



► **SAY IT OUT LOUD** — this page is **GOOD**. And it is a **SINGLE COLUMN**, because that is the only shape you know how to make.

THIS IS WHERE YOU WILL BE IN NINETY MINUTES



► **THE POINT** — everything that moves when you resize this does so without a single line that mentions a screen size.

ACT 1 · THE BROWSER ALREADY DID THIS

NOTHING YOU HAVE WRITTEN SO FAR HAS PLACED ANYTHING

Every page you have made was laid out by a rule you were never told. It is **two sentences long**.

A BLOCK BOX

Takes the full width offered; the next one starts underneath.

```
p h1 div section li
```

AN INLINE BOX

Takes the width of its content; the next one continues on the line.

```
a span strong em img
```

LOOK AT THE BOXES THE BROWSER MADE

demos/01-normal-flow — no layout CSS at all↶ reloadopen ↷

☒ outline every box ☐ show the line boxes ☐ show the containing block

■ block box

■ inline box

■ line box

■ containing block

Normal flow

This paragraph is a **block box**. It takes the whole width it is offered, and the next block box starts underneath it. Nobody asked for that: it is what the browser does when you say **nothing**.

Inline content flows

Inside a block box, text and **inline boxes** are poured into **line boxes** — horizontal strips as wide as the block, filled one after another until the words run out. An **inline box** is exactly as wide as its content, and an  sits on the text baseline like a very large letter.

- A list item is a block box too.
- So they stack.

Everything on this page was laid out before you wrote a single layout rule.

Hover an element in the page above.

Hover any element in the page above to measure it. Nothing here is drawn by hand — the strips and the frame are positioned from `getClientRects()`.

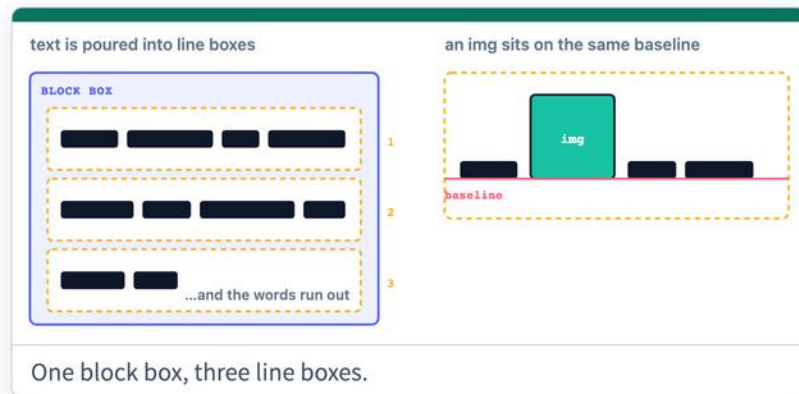
► **HANDS-ON** `demos/01-normal-flow/` — Turn on the outlines. Blue is a block box, orange is an inline box. Then hover anything and read what it was offered.

5

INLINE CONTENT IS POURED, NOT STACKED

Inside a block box, text and inline boxes are poured into **line boxes** — horizontal strips as wide as the block, filled one after another until the words run out.

That is why a line-height changes the height of a paragraph, and why an `img` in a sentence sits on the text baseline like a very large letter.



EVERY BOX IS MEASURED AGAINST ANOTHER BOX

containing block The box your box's sizes and offsets are resolved against.

in normal flow Your nearest block-level ancestor's **content box**.

why you care `width: 50%` is 50% *of that*. So is `left: 10%`.

Change which box that is, and every percentage in the element changes meaning without you touching it. **Act 3 is entirely about changing it.**

```
1 .parent {  
2   width: 600px;    /* the containing block */  
3 }  
4  
5 .child {  
6   width: 50%;      /* 300px */  
7 }
```

SO THIS IS WHY LAST WEEK'S `width` DID NOTHING

WEEK 2, AND IT DID NOTHING

```
1 nav a {  
2   width: 160px;  
3   height: 44px;  
4 }
```

You wrote it. It was correct CSS. Nothing happened.

THE MISSING SENTENCE

```
1 nav a {  
2   display: inline-block;  
3   width: 160px;  
4   height: 44px;  
5 }
```

An inline box is sized by its content. Always was.



ACT 2 · DISPLAY

display DECIDES WHAT KIND OF BOX

It is not decoration and it is not visibility. It answers one question: *what sort of box does this element generate?*

block

Full width offered · stacks · width, height and all margins work.

inline

Content width · flows in a line · width, height and vertical margin ignored.

inline-block

Content flow, block sizing. Both, and one famous problem.

none

No box at all. Not "invisible" — absent.

THE SAME THREE DECLARATIONS, THREE TIMES

demos/02-display/01-inline-wont-size [↶ reload](#) [open ↷](#)

THE SAME DECLARATIONS, THREE TIMES

display: inline
Text before the link and text after it, on the same line.

display: inline-block
Text before the link and text after it, on the same line.

display: block
Text before
the link

► **HANDS-ON** `demos/02-display/01-inline-wont-size/` — Ask the room for the first link's height before you read the measured number.

THREE CARDS AT 33.333% DO NOT FIT IN 100%

NOTHING IS WRONG WITH THIS CSS

```
1 .card {  
2   display: inline-block;  
3   width: 33.333%;  
4   margin: 0;  
5 }  
6  
7 /* 3 × 33.333% = 100%  
8    so why did the third  
9    one drop to a new  
10   line?                */
```

THE CULPRIT IS IN THE HTML

```
1 <div class="card">one</div>  
2 <div class="card">two</div>  
3 <div class="card">three</div>  
4 <!-- the newline and the indentation  
5      between the tags are WORDS -->
```

An inline-block is an inline box. Inline boxes are separated by whitespace.

► **HANDS-ON** [demos/02-display/02-inline-block-gaps/](#) — Four fixes, and only one of them is a good idea.

FOUR WAYS TO MAKE SOMETHING DISAPPEAR

`display: none`

No box. No space, no tab stop, not read aloud. The honest "gone".

`visibility: hidden`

Invisible, **keeps its space**. Not focusable, not read aloud.

`opacity: 0`

Only transparent. Still takes space, still focusable, still read aloud, **still clickable**.

► **HANDS-ON** [demos/02-display/03-none-vs-hidden/](#) — Choose `opacity: 0`, then click where the button should be and watch the counter.

ACT 3 · POSITION

TAKING A BOX OUT OF THE CONVERSATION

Normal flow is a negotiation between siblings. `position` is how one box *leaves* it — and the cost is that nothing reserves its space any more.

static

The default. In flow. Offsets ignored.

relative

In flow, keeps its space, moves visually.

anchors its children.

absolute

Out of flow. Measured from the nearest positioned ancestor.

fixed

Out of flow. Measured from the viewport. Does not scroll.

sticky

In flow, then pins at a threshold. The one you will ship.

ALL FIVE, ON ONE SCROLLING PAGE

[demos/03-position/02-five-values](#)[↶ reload](#)[open ↷](#)

[static](#)[relative](#)[absolute](#)[fixed](#)[sticky](#)

THE DASHED FRAME IS THE CONTAINER. IT IS `position: relative`.

Text above the box.

where it would be in normal flow

the box

`position: static`

still in normal flow?	yes – this IS normal flow
offsets measured from	nothing, top and left are ignored entirely.
when you scroll	scrolls with the page
container height now	277px
computed position	static

The default. Every element on every page you have written so far.
Note that top/left do nothing here – offsets need a position first.

► **HANDS-ON** [demos/03-position/02-five-values/](#) — Start on `relative` and point at the dashed ghost: **THAT SPACE IS STILL RESERVED.**

absolute — MEASURED FROM WHAT, EXACTLY?

- 1 **Walk up**
from the element, parent by parent.
- 2 **Stop**
at the first ancestor whose `position` is not `static`.
- 3 **Measure**
the offsets from that ancestor's padding box.
- 4 **If there is none**
, fall back to the page itself.

That is the whole algorithm. Step 4 is where everybody gets hurt, because it **always succeeds** — it never errors, it just anchors somewhere you did not mean.

```
1 .card {  
2   position: relative; /* ← the anchor */  
3 }  
4  
5 .badge {  
6   position: absolute;  
7   top: -10px;  
8   right: -10px;  
9 }
```

MISCONCEPTION 1 · THE BADGE THAT FLEW AWAY

[demos/03-position/01-absolute-adrift](#)[↶ reload](#)[open ↷](#)

☐ **position:** relative on the middle card ☐ show the ancestor chain

THE BADGE BELONGS TO THE MIDDLE CARD

the page

one

two

three

NEW

The badge landed 525px away from the card it belongs to.

```
.badge { position: absolute; top: -10px; right: -10px; }  
- unchanged. The badge never moves in this demo.
```

walking up from the badge, looking for a positioned ancestor:

```
<div#card>          static  
<div#row>           static  
<div.page-root>     relative ← this one anchors the badge  
<body>              static  
<html>              static
```

The walk went straight past the card – it is `static`, so it does not count – and stopped at the page. The offsets are measured from THERE.

"If it aligns to the page instead of the box you expected, you probably forgot position: relative on the ancestor."

► **HANDS-ON** [demos/03-position/01-absolute-adrift/](#) — The badge's CSS never changes. Turn on the ancestor chain.

KEEP THIS ONE. IT IS A DIAGNOSTIC, NOT A DEFINITION.

If it aligns to the page instead of the box you expected, you probably forgot `position: relative` on the ancestor.

FROM THIS COURSE, BEFORE YOU WERE IN IT

AND ITS TWO COMPANIONS

Absolute elements won't stretch the parent's height
— parents may collapse.

fixed inside a transformed ancestor behaves like `absolute`. Avoid transforming high-level wrappers.

z-index, THE MODEL EVERYONE HAS

Bigger number in front. Default is 0. Only works on positioned elements.

All three of those are true, and together they are still not enough to explain the next slide.

```
1 .dropdown {  
2   position: absolute;  
3   z-index: 999;  
4 }  
5  
6 .header {  
7   position: sticky;  
8   z-index: 1;  
9 }
```

999 > 1. So the dropdown is in front. Obviously.

MISCONCEPTION 2 · WHEN 999 LOSES TO 1

demos/04-stacking/01-z-index-lies ↻ reload open ↗

dropdown z-index ☐ opacity: 0.99 on the panel ☐ transform: translateZ(0) on the panel ☐ filter: saturate(1) on the panel ☐ will-change: transform on the panel ☐ isolation: isolate on the panel

THE GREEN DROPDOWN HAS Z-INDEX 999. THE PINK HEADER HAS Z-INDEX 1.

Menu

I am z-index: 999

The dropdown is on top, as asked.

```
.panel { }           ← creates no stacking context
.dropdown { z-index: 999 }
.header { z-index: 1 }
```

Both are in the same stacking context – the root – so the two numbers are genuinely comparable, and the bigger one wins.

Now tick any box above. None of them mentions layering.

► **HANDS-ON** [demos/04-stacking/01-z-index-lies/](#) — Ask what `opacity: 0.99` will do. Then tick it. Then let someone try 99999.

A STACKING CONTEXT IS A SEALED BOX

When an element creates one, its children's `z-index` values are only ever compared **with each other**. Against anything outside, what competes is *the parent's own* `z-index`.

So `999` was never being compared with `1`. It was being compared with its siblings, inside a box whose own `z-index` is `auto` — and `auto` loses to `1`.

Raising the number cannot help. It is the wrong argument.

`position`

`opacity`

`transform`

`filter`

`will-change`

`isolation`

...that is not `static`, plus any `z-index`

anything below `1`

anything other than `none`

anything other than `none`

naming any of the above

`isolate` — the only one that is deliberate

LIVE CODING · BREAK ONE

- 1 **Type the badge — absolute, negative offsets.**
Watch it fly to the corner.
- 2 **Ask for the fix before you give it.**
Let someone say "relative on the card".
- 3 **Add it, and point out that nothing else moved.**
- 4 **Then break it again — put `opacity: 0.99` on the row.**

► **SEVEN MINUTES.** Build it from an empty file. If the room is with you after step 3, do step 4; if not, stop at three.

ACT 4 · OVERFLOW AND HEIGHT

WHEN THE CONTENT DOES NOT FIT

visible

The default. Content paints outside the box, over whatever is under it.

hidden

Clipped — and it becomes a **scroll container**. The content is still findable by `Ctrl-F` and by `Tab`.

auto

A scrollbar only when one is needed. The one you usually want.

► **HANDS-ON** <demos/05-overflow/01-overflow-values/> — Pick `hidden`, then press `Ctrl-F` in the frame and search for **BURIED**.

A SCROLL CONTAINER IS ALSO WHAT **sticky** STICKS INSIDE

The most common reason a perfectly correct `position: sticky` does nothing is **an overflow somebody added to an ancestor for an unrelated reason**.

A sticky element can only travel inside its own scroll container. If that container is no taller than the element, it never has anywhere to stick.

HOW TO FIND IT

Walk up the ancestors in DevTools looking for `overflow` that is not `visible`. It is almost always three or four levels up.

MISCONCEPTION 5 · height: 100% OF WHAT?

demos/05-overflow/02-height-100

reload

open

☐ height:100% on <html> ☐ height:100% on <body> ☐ height:100% on .wrapper instead: min-height on the box itself

THE BLUE BOX SAYS height: 100%. IT ALWAYS HAS.

```
html
  body
    .wrapper
      height: 100%
```

```
.target { height: 100% } ← never changes in this demo
```

the chain above it:

html	height: auto	157px	← auto: as tall as its children
body	height: auto	118px	← auto: as tall as its children
.wrapper	height: auto	80px	← auto: as tall as its children

► **HANDS-ON** [demos/05-overflow/02-height-100/](#) — Turn the three switches on left to right. Nothing happens until the last one.

ASK NOBODY: USE A VIEWPORT UNIT

NEEDS THREE ANCESTORS TO COOPERATE

```
1 html, body { height: 100%; }  
2 .wrapper { height: 100%; }  
3 .hero { height: 100%; }
```

NEEDS NOBODY

```
1 .hero {  
2   min-height: 100dvh;  
3 }
```

`dvh` is the *dynamic* viewport height — the one that stays correct when a phone browser slides its toolbar away.

ACT 5 · FLEXBOX

ONE DIMENSION, DONE PROPERLY

It's designed for components — navbars, cards, toolbars, forms — rather than full page grids.

FROM THIS COURSE, BEFORE YOU WERE IN IT

A **row or a **column**, with real control over how the space between and around the items is distributed. That is the entire scope, and it is why it is easy.**



THE SAME NAVBAR, FOUR WAYS

[demos/06-flex/01-why-flex](#)[↶ reload](#)[open ↷](#)

[normal flow](#)[inline-block](#)[float](#)[flex](#)

THE MARKUP NEVER CHANGES. ONLY THE STYLESHEET DOES.

Ruppin FE
Home
Work
About
Sign in

This paragraph comes after the bar in the markup.

```
/* nothing at all */
```

```
bar height      187px  
links on one row no – they are on 4 rows  
"Sign in" at the end yes
```

Every `<li>` is a block box, so they stack. This is not a bug – it is the default, and it is the right default for a list.

The question the rest of act 5 answers: how do you ask for a row?

► **HANDS-ON** [demos/06-flex/01-why-flex/](#) — Run the `float` version and ask why the gold bar has no height.

TWO AXES, AND THEY MOVE

main axis The direction items are laid out. Set by `flex-direction`.

cross axis Perpendicular to it. Always.

justify-content Distributes along the **main** axis.

align-items Aligns along the **cross** axis.

Neither of those two is "horizontal" or "vertical". They are **main** and **cross**, and which is which is a decision you made one line earlier.

Where the axes point

FLEX-DIRECTION	MAIN	CROSS
<code>row</code>	horizontal →	vertical ↓
<code>row-reverse</code>	horizontal ←	vertical ↓
<code>column</code>	vertical ↓	horizontal →
<code>column-reverse</code>	vertical ↑	horizontal →

MISCONCEPTION 3 · WATCH THE AXIS MOVE

demos/06-flex/playground — every property, live reload open

THE CONTAINER

flex-direction
row

flex-wrap
nowrap

justify-content — horizontal now
flex-start

align-items — vertical now
stretch

align-content (needs wrap)
normal

gap
10px

THE SELECTED ITEM

flex-grow
0

flex-shrink

main axis →

cross axis ↓

1 2 3 — a taller item, with more words in it 4 5

```
.container {  
  display: flex;  
  gap: 10px;  
}
```

container 1278 × 240 main axis is horizontal
item 1 56 × 216 ← selected
item 2 56 × 216
item 3 190 × 216
item 4 56 × 216
item 5 56 × 216

► **HANDS-ON** [demos/06-flex/playground/](#) — Switch to `column`. The arrows turn, and the labels rewrite themselves.

LIVE CODING · BREAK TWO

1 Ask for the goal — brand left, links right, one line, centred.

2 Take suggestions and type them.

Wrong ones too — undo is free.

3 Land on `display: flex + align-items: center + gap`.

4 Then `margin-inline-start: auto`, and ask what it did.

► **EIGHT MINUTES.** This is the exact component they build in the lab. Do not skip it.



CONTAINER PROPERTIES · ITEM PROPERTIES

ON THE CONTAINER

`flex-direction` · `flex-wrap` · `justify-content` ·
`align-items` · `align-content` · `gap`

Controls the flow and the alignment of all the items at once.

ON AN ITEM

`flex-grow` · `flex-shrink` · `flex-basis` (and the
`flex` shorthand) · `align-self` · `order`

Controls the size and the alignment of one item.

Start with `flex-basis` to set the natural size,
then tweak `flex-grow` and `flex-shrink`.

THE THREE NUMBERS

[demos/06-flex/02-grow-shrink-basis](#) [↶ reload](#) [open ↷](#)

the default (0 1 auto)

all three: flex: 1

all three: flex: auto

grow 2 / 1 / 1

a sidebar: 200px + flex: 1

item 1

flex-grow
0

flex-shrink
1

flex-basis
auto

item 2

flex-grow
0

flex-shrink
1

flex-basis
auto

item 3

flex-grow
0

flex-shrink
1

flex-basis
auto

THREE ITEMS IN A 620PX RAIL, WITH TWO 10PX GAPS

one

two

three — the item with a good deal more text in it

container	596px
– two gaps	20px
– Σ basis	403px
free space	173px

► **HANDS-ON** [demos/06-flex/02-grow-shrink-basis/](#) — The panel does the arithmetic and then prints the browser's measured widths beside it. They match.

THE SHORTHAND YOU WILL ACTUALLY READ

flex: 1 `1 1 0%` — basis zero, so content is ignored and widths come out equal.

flex: auto `1 1 auto` — basis is the content, so wider items stay wider.

flex: none `0 0 auto` — never grows, never shrinks. For the logo.

That first difference is most of the confusion about `flex` in other people's code.

```
1 .card {  
2   /* every card the same width */  
3   flex: 1;  
4 }  
5  
6 .card {  
7   /* wider cards stay wider */  
8   flex: auto;  
9 }
```

RESPONSIVE, WITH NOTHING THAT MENTIONS A SCREEN

THE WHOLE CARD ROW

```
1 .card-row {  
2   display: flex;  
3   flex-wrap: wrap;  
4   gap: 1.5rem;  
5 }  
6  
7 .card {  
8   flex: 1 1 17rem; /* I want 17rem.  
9                     If you cannot,  
10                    take a line. */  
11 }
```

Three across at 1280. Two at 900. One at 375. **No media query, no breakpoint, no arithmetic that has to add up to 100 by hand.**

`flex-wrap` says items may take a new line rather than overflow; `flex-basis` says how much room each one wants before it gives up. Between them they replace most of what week 4 is about.

MISCONCEPTION 4 · `margin: auto` IS NOT A LAYOUT SYSTEM

THE HABIT

Reaching for `margin: 0 auto`, then a negative margin, then another one, until it looks right at the width you happen to be at.

THE ONE REAL USE, IN FLEX

An auto margin **absorbs the free space**. In a flex row that makes it a positioning tool:

`margin-inline-start: auto` pushes one item, and everything after it, to the end.

AND THE OTHER ONE

`margin-block-start: auto` in a flex *column* pins an item to the bottom. That is how the pricing buttons line up tonight.

THE ONE THAT WILL COST YOU AN AFTERNOON

[demos/06-flex/03-min-width-auto](#) [↶ reload](#) [open ↷](#)

☐ put a long unbreakable string in the content ☐ min-width: 0 on .content ☐ + text-overflow: ellipsis ☐ flex-shrink on .content  1

A 150PX SIDEBAR AND A flex: 1 CONTENT COLUMN

sidebar
150px

The content column is flex: 1, so it should take what is left.

The row fits its container.

```
container inner width  1498px
the flex row wants    1498px
.content min-width    auto
.content flex-shrink   1
```

Nothing is wrong yet. The content is ordinary text, which can wrap, so its minimum content width is small and flex-shrink has room to work.

Tick the first box to give it something that cannot wrap.

► **HANDS-ON** [demos/06-flex/03-min-width-auto/](#) — Turn on the long string. Now try to fix it with `flex-shrink`. Drag it to 10. Nothing.

order AND row-reverse — AND THE WARNING

Both change the **visual** order of items without touching the HTML. Both are genuinely useful, and both are a trap.

Tab order, screen-reader order and copy-paste order all follow the **DOM**. Reorder visually and you have made three orders that disagree — and only one of them is the one you tested.

Rule: if the order matters, fix the HTML. Use these for presentation, never for meaning.

align-self

Overrides `align-items` for one item.

order

Default 0. Lower comes first. Negative is allowed.

align-content

Only does anything when `flex-wrap: wrap` is on.

ACT 6 · ARCHAEOLOGY

TWO MINUTES ON `float`, BECAUSE YOU WILL MEET IT

demos/07-archaeology — five ways, 1998 to 2017

↻ reload

open ↗

1 · tables (1998)

2 · inline-block (2007)

3 · absolute (2005)

4 · float (2010)

5 · flex (2017)

THE SAME THREE COLUMNS. THE MARKUP NEVER CHANGES.

Column one, with a sentence in it so the boxes have something to be.

Column two.

Column three, which is the longest of the three by some way.

A paragraph after the columns.

```
.cols { display: table; width: 100%; border-spacing: 5px }  
.col { display: table-cell; width: 33.33% }
```

What it cost: the layout lives in the markup. A screen reader announces a table with rows and cells to someone who is trying to read an article, and the page cannot be rearranged for a narrow screen at all — a table is a table at every width.

Tables are for tabular data. That has never changed.

► **HANDS-ON** `demos/07-archaeology/` — Click through in order and read the "what it cost" line. Then stop.

WHAT `float` IS STILL FOR

Wrapping text around an image. That is what it was invented for in 1996, and it is still the only thing it does that nothing else does.

Today, using floats for overall page layout is discouraged.

If you find `float: left` on a column in a stylesheet you have inherited, you are looking at 2010. Replace it with a flex row and delete the clearfix.

```
1 .pull-quote {  
2   float: inline-start;  
3   width: 14rem;  
4   margin-inline-end: 1rem;  
5 }
```

Legitimate. Text flows around it, which is exactly the job.

THREE QUESTIONS BEFORE YOU LEAVE

1. A badge with `position: absolute; top: 0; right: 0` lands in the corner of the page instead of the corner of its card. What is missing, and on which element?
2. A dropdown has `z-index: 999`. A header has `z-index: 1`. The dropdown is behind. Give me one reason — and tell me why raising 999 will not help.
3. `flex-direction: column`. Which property now centres the items left-to-right, and which one distributes them top-to-bottom?

WEEK 03 · NEXT

THE LAB, AND WHAT IS DUE

IN THE LAB, NOW

`week-03/` : a landing page with a responsive navbar, a hero and a card row. The markup is given — you write the layout.

Flexbox only. No Grid, no float, one media query.

WARM UP FIRST

Ten minutes on `warmup/` — the gold navbar, three declarations. No git, no submission, no grade. Then start the assignment.

BEFORE NEXT WEEK

Nothing to install. Next week the layout gets a second dimension: **CSS Grid**, and the rule for choosing between the two.